

Exercises on arrays (and pointers)

Implement the following functions together with a testing main.

1. merge .

Define a function that, given two integer arrays `a` and `b` (of certain length), which are assumed to contain values in increasing order, merges them into a third sorted array `c`, which is assumed large enough

```
void merge(int a[], int la, int b[], int lb, int c[]);
```

2. readline

Define a function that using the library function

```
int getchar();
```

reads a line from the standard input into an array and appends a final `'\0'` at the end.

The function stops after reading the `'\n'` character, the `max` characters limit is reached, or `getchar` returns `EOF` (corresponding to pressing `ctrl-D`)

```
int readLine(char line[], int max);
```

3. palyndrome

Define a function that given a string (an array of chars ending with `\0`) verifies if it is palyndrome (`_Bool` is an integer subtype with values `0` and `1`)

```
_Bool palyndrome(char *s);
```

4. integer extraction.

Write a program that given a string extracts the sequence of digits in that string and returns the corresponding numerical value.

For example, if the string is `"ajh7PH1e:;()1k9"` then the function return `719`. If the string doesn't contain any digits then the function returns `-1`.

```
int extractInt(char *s);
```

Suggestion. The function `sscanf` is similar to `scanf` except for reading data from a string (the new first parameter); it can be used to easily convert a string of digits into a number. For example

```
char str[] = "30";
int n;
sscanf(str, "%d", &n); // n holds 30
```

5. strcmp

Redefine the library function `strcmp` that compares lexicographically (according to the ASCII values) two strings, returning `-1`, `0`, `1` depending on whether the 1st string is less than, equal to, or greater than the 2nd string, respectively.

```
int myStrcmp(char *s1, char *s2);
```

6. countwords.

Write a function that counts how many words are contained in a given string; a word is an arbitrary sequence of latin letters. For example, the string `"!()df15ci Ao"` contains three words

```
int countwords(char s[]);
```

7. first occurrence.

Write a function that verifies if the string `s2` is contained in `s1`, returning the pointer to the first occurrence of the `s2` in `s1`. The function returns `NULL` (null pointer value) if `s1` does not contain `s2`.

```
char *firstoccurrence(char *s1, char *s2);
```

8. replaceall.

Write a function that replaces all the occurrence of character `a` with `b` in a given string. The function returns the number of replacements.

```
int replaceall(char *s1, char a, char b);
```

9. mode.

Write a program that reads an arbitrary sequence of grades (between `18` and `30`) and calculates the mode, that is, the most frequent value. If the sequence of values is multi-modal, the program should print all the modal values.

The reading should end when `EOF` is reached (`ctrl-D`) or an invalid value is entered.

10. base conversion.

Write a program that reads an integer value `n` and a new base `b` (between `2` and `16`) and prints the result of the conversion of `n` in base `b`. For example, if the input is

```
31 16
```

the output should be

```
1F
```

Hint: use the array

```
char symbol[ ] = {'0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'A', 'B', 'C', 'D', 'E', 'F'};
```

which contains the set of all possible digits, each in its respective position (`'0'` is in position `0`, `'1'` is in position `1` etc.). Remember that `n % b` (a value between `0` and `b - 1`) is the least significant "digit" of the conversion of `n` while `n / b` "cuts off" the least significant digit.

11. anagrams.

Write a program that reads two words and checks whether they are anagrams. Here is an example of execution

```
first word:
reverse
second word:
verrees
they are anagrams!
```

Use the library function

```
int getchar()
```

to read the words; the reading stops when any non-letter is entered